Lecture 11 · Part II · Extended notes

# Autoencoders and representation learning

Dataset: Fashion-MNIST, 12,000 training images

**Why these notes exist**

This lecture sits outside Géron, the course's primary text. For it *these notes are the primary source*, and it is examined on exactly the same terms as the textbook lectures. Everything in the slide deck is here, with the arguments written out at length rather than compressed onto a slide; the numbers are the ones the accompanying notebook prints, and every one of them is a measurement on the dataset named above rather than a figure quoted from a paper.

## Contents

**1 Where we are** **2**

**2 The problem** **3**

2.1 The data, and the one rule about labels . . . . . 3

**3 What an autoencoder is** **3**

3.1 The bottleneck is a constraint on what the network can reach . . . . . 4

**4 Derivation: a linear autoencoder recovers the principal subspace** **4**

4.1 The model, with the shapes tracked . . . . . 4

4.2 The subspace is unique; the code is not . . . . . 5

4.3 What the result buys you . . . . . 6

4.4 Checked numerically . . . . . 6

**5 The undercomplete autoencoder** **6**

5.1 The model . . . . . 6

5.2 Run it at $d = 32$ . . . . . 7

5.3 The code size, swept . . . 7
5.4 What the loss sees, and what it does not . . . 8

**6 The code as a representation** **8**
6.1 The code as a picture . . . 9
6.2 What a representation is for . . . 9
6.3 Similar items: nearest neighbours . . . 9
6.4 The bill . . . 10
6.5 Groups: $k$-means on the code . . . 10

**7 The denoising autoencoder** **11**

**8 Anomaly detection by reconstruction error** **11**
8.1 The three failure conditions, in one place . . . 12

**9 The variational autoencoder** **12**

**10 Where this leads** **13**

**11 The notebook** **13**

**12 Design exercise: "see similar" on a fashion marketplace** **14**

**13 Exercises** **15**

---

# 1 Where we are

Every network of Part II so far needed a label for every image. Lecture 7 built a first network and said what a layer computes; Lecture 8 wrote the training loop out in five lines. Each of them was scored by a label it was given, and so were the systems of Lecture 9 and Lecture 10: a harmful-content detector designed end to end, and a model audited for what it relies on and for whom it fails.

This lecture removes the label. A network that is asked to *reproduce its input* through a code too small to hold it has to learn what the inputs have in common, and the code it learns is a description of the input that nobody wrote down. The lecture is about that code: what it is in the one case where the answer is closed-form, how it is trained in general, and what it is good for.

**What you should be able to do afterwards.**

- Write down an autoencoder's objective and say what stops it learning the identity.

- Prove that a linear autoencoder with squared loss cannot do better than the projection onto the principal subspace, and reaches it.
- Say what the non-linear model buys over that floor, and when a trained approximation loses to an exact one.
- Read a learned code as a representation: measure it on retrieval and clustering against the baseline it was built from, and say what it bought.
- State what the denoising corruption does, and why reconstruction error is an anomaly score only for some anomalies.

## 2 The problem

A marketplace holds millions of product photographs uploaded by its sellers. The category field is free text, often empty, sometimes wrong. The product team wants three things from the photographs alone: given one garment, the ten most like it, in milliseconds, across the whole catalogue; a first structure for a catalogue that has none; and the uploads that are not what the catalogue holds, flagged before a shopper sees them. None of these comes with a label.

### 2.1 The data, and the one rule about labels

Fashion-MNIST stands in for the catalogue: seventy thousand greyscale images of garments, 28 by 28 pixels, ten classes. The split is the one Lecture 7 and Lecture 8 used, so the numbers compare: a validation set of 5,000 carved out of the training images from a fixed seed, the first 12,000 of the rest as the fit set, and the official 10,000 test images as the catalogue.

| Quantity | Count |
|---|---|
| Images to fit on | 12,000 |
| Validation images | 5,000 |
| Test images, the catalogue | 10,000 |
| Pixels per image | 784 |
| Classes | 10 |

**The labels are loaded, and used for exactly one thing**

Scoring. Nothing in this lecture is fitted on a label. Precision at ten, the clusters' purity and the anomaly detector's AUROC all read the labels *after* the fact, to say whether a representation learned without them means anything. The one other use is in Section 8, where a label is used to *remove* a class from the fit set.

## 3 What an autoencoder is

**Definition 3.1** (Autoencoder)**.** *An **encoder** $f_{enc} : \mathbb{R}^n \to \mathbb{R}^d$ and a **decoder** $f_{dec} : \mathbb{R}^d \to \mathbb{R}^n$, trained together so that the composition reproduces its input:*

$$\mathbf{z} = f_{enc}(\mathbf{x}), \qquad \hat{\mathbf{x}} = f_{dec}(\mathbf{z}), \qquad L = \frac{1}{m}\sum_{i=1}^{m} \left\| \mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)} \right\|_2^2 .$$

*The vector* $\mathbf{z}$ *is the* ***code****;* $d$ *is its size.*

The loss is the squared loss of least squares with the input standing in for the label, and the training loop is the five-line loop of Lecture 8 with line 3 comparing the output with the input. Nothing in the loop knows there is no label.

### 3.1 The bottleneck is a constraint on what the network can reach

A regulariser — dropout, a penalty on the size of the weights, randomly altered copies of the training inputs — works by shrinking the set of functions the network can reach, so that fitting the training data no longer means memorising it. An autoencoder needs the same thing, for a sharper reason. With $d = n$ the identity is reachable, the loss is zero, and nothing was learned. Three constraints are in common use.

| Constraint | What it forbids | Name |
|---|---|---|
| $d \ll n$ | copying: 784 numbers cannot pass through 32 | undercomplete |
| corrupt the input, keep the target clean | copying: what comes in is not what must go out | denoising |
| penalise the code | using every coordinate for every image | sparse |

This lecture uses the first two. The first is the one the mathematics is about.

## 4 Derivation: a linear autoencoder recovers the principal subspace

Make the encoder and the decoder linear, keep the squared loss, and train by gradient descent. What does the network learn? The claim: exactly what Lecture 6 computed with the SVD. The best such network reconstructs through the projection onto the span of the leading $d$ right singular vectors, and the error it leaves is the tail of the spectrum, $\sum_{j>d} \sigma_j^2$.

### 4.1 The model, with the shapes tracked

One photograph per row, as in Lecture 6, and the data centred on its mean: $\tilde{\mathbf{X}} \in \mathbb{R}^{m \times n}$. The encoder is a matrix $\mathbf{A} \in \mathbb{R}^{n \times d}$, the decoder a matrix $\mathbf{B} \in \mathbb{R}^{d \times n}$, and

$$\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{A} \in \mathbb{R}^{m \times d}, \qquad \hat{\mathbf{X}} = \mathbf{Z}\mathbf{B} = \tilde{\mathbf{X}}\mathbf{A}\mathbf{B}, \qquad E(\mathbf{A}, \mathbf{B}) = \left\|\tilde{\mathbf{X}} - \tilde{\mathbf{X}}\mathbf{A}\mathbf{B}\right\|_F^2.$$

There is no orthogonality and no constraint on $\mathbf{A}$ or $\mathbf{B}$ at all: gradient descent sees $2nd$ free numbers. The Frobenius norm is the one of Lecture 6, $\|\mathbf{M}\|_F^2 = \sum_{ij} m_{ij}^2$.

**Seven steps**

**1. The objective.** $E(\mathbf{A}, \mathbf{B}) = \|\tilde{\mathbf{X}} - \tilde{\mathbf{X}}\mathbf{A}\mathbf{B}\|_F^2$, with $\mathbf{A}$ of shape $n \times d$ and $\mathbf{B}$ of shape $d \times n$.
**2. Every reconstruction lies in one subspace.** Row $i$ of $\hat{\mathbf{X}}$ is $\mathbf{z}^{(i)}\mathbf{B}$: a combination of the $d$ rows of $\mathbf{B}$, with the $d$ entries of the code as coefficients. So

$$\hat{\mathbf{x}}^{(i)} \in S := \text{rowspace}(\mathbf{B}) \subseteq \mathbb{R}^n, \qquad \dim S \le d.$$

Whatever the encoder does, the decoder can only put a reconstruction somewhere in $S$. This is the only place the bottleneck enters the argument, and it enters as a dimension count.
**3. The closest point of a subspace is the projection.** Let $\mathbf{P}_S$ be the orthogonal projector onto $S$. For any $\mathbf{y} \in S$,

$$\tilde{\mathbf{x}} - \mathbf{y} = (\tilde{\mathbf{x}} - \mathbf{P}_S\tilde{\mathbf{x}}) + (\mathbf{P}_S\tilde{\mathbf{x}} - \mathbf{y}),$$

the first piece orthogonal to $S$ and the second inside it, so by Pythagoras

$$\|\tilde{\mathbf{x}} - \mathbf{y}\|^2 = \|\tilde{\mathbf{x}} - \mathbf{P}_S\tilde{\mathbf{x}}\|^2 + \|\mathbf{P}_S\tilde{\mathbf{x}} - \mathbf{y}\|^2 \;\ge\; \|\tilde{\mathbf{x}} - \mathbf{P}_S\tilde{\mathbf{x}}\|^2.$$

**4. Sum over the rows.** Apply step 3 to every row with $\mathbf{y} = \hat{\mathbf{x}}^{(i)}$. With $\mathbf{W}$ an orthonormal basis of $S$, so that $\mathbf{P}_S = \mathbf{W}\mathbf{W}^\top$,

$$E(\mathbf{A}, \mathbf{B}) \;\ge\; \sum_i \|\tilde{\mathbf{x}}^{(i)} - \mathbf{P}_S\tilde{\mathbf{x}}^{(i)}\|^2 = \left\|\tilde{\mathbf{X}} - \tilde{\mathbf{X}}\mathbf{W}\mathbf{W}^\top\right\|_F^2.$$

If $\dim S < d$, pad $\mathbf{W}$ to $d$ orthonormal columns: a larger subspace can only lower the right-hand side. The right-hand side is exactly Lecture 6's objective $E(\mathbf{W})$, with $\mathbf{W}^\top\mathbf{W} = \mathbf{I}_d$.
**5. Recall Lecture 6.** With $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ and $\sigma_1 \ge \sigma_2 \ge \dots$, over all $\mathbf{W}$ with orthonormal columns,

$$\min_{\mathbf{W}} \left\|\tilde{\mathbf{X}} - \tilde{\mathbf{X}}\mathbf{W}\mathbf{W}^\top\right\|_F^2 = \sum_{j>d} \sigma_j^2, \qquad \text{attained at } \mathbf{W} = \mathbf{V}_d,$$

the first $d$ columns of $\mathbf{V}$. (Pythagoras turns the error into $\|\tilde{\mathbf{X}}\|_F^2 - \text{tr}(\mathbf{W}^\top\mathbf{C}\mathbf{W})$, the SVD diagonalises $\mathbf{C} = \tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$, and $d$ units of weight spent on the largest $\sigma_j^2$ maximise the trace.)
**6. The bound is attained.** Steps 4 and 5 give $E(\mathbf{A}, \mathbf{B}) \ge \sum_{j>d} \sigma_j^2$ for every $\mathbf{A}$ and $\mathbf{B}$. Choose $\mathbf{A} = \mathbf{V}_d$ and $\mathbf{B} = \mathbf{V}_d^\top$: then $\mathbf{A}\mathbf{B} = \mathbf{V}_d\mathbf{V}_d^\top$ and $E = \sum_{j>d} \sigma_j^2$. The encoder computes the principal components and the decoder puts them back; the lower bound is reached, so it is the minimum.
**7. The result.**

$$\min_{\mathbf{A},\mathbf{B}} E(\mathbf{A}, \mathbf{B}) = \sum_{j>d} \sigma_j^2,$$

and at any optimum the reconstruction is $\tilde{\mathbf{X}}\mathbf{V}_d\mathbf{V}_d^\top$: step 3 is an equality only when every $\hat{\mathbf{x}}^{(i)}$ is the projection, and step 5 picks out the principal subspace when the $\sigma_j^2$ around position $d$ are distinct.

## 4.2 The subspace is unique; the code is not

For any invertible $d \times d$ matrix $\mathbf{T}$,

$$(\mathbf{A}\mathbf{T})\,(\mathbf{T}^{-1}\mathbf{B}) = \mathbf{A}\mathbf{B},$$

so the encoder $\mathbf{AT}$ with the decoder $\mathbf{T}^{-1}\mathbf{B}$ gives the same reconstruction and a different code, $\mathbf{ZT}$. Gradient descent lands on *one* of these: the principal subspace in some basis, not the principal components themselves. That is the claim to check numerically — the error and the subspace, not the coordinates.

### 4.3 What the result buys you

- **A floor you can compute before training.** $\sum_{j>d} \sigma_j^2$ comes from one SVD. A linear autoencoder that reports less has a bug; one that reports much more has not converged.
- **A definition of what the non-linear model is for.** Whatever it gains over the floor is the part of the error no projection can remove: curvature in the data that a subspace cannot follow.
- **The bottleneck is the whole constraint.** At $d = n$ the tail is empty, the floor is zero, the identity is reached, and the code is a rotation of the pixels. Nothing learned, no error raised.
- **The code is a representation only up to a change of basis.** Distances in it are the distances between principal components under $\mathbf{T}$. Retrieval in a learned code is retrieval in a metric nobody chose.

### 4.4 Checked numerically

The notebook computes the floor from the SVD of the centred fit set, then trains two bias-free linear layers $784 \to 32 \to 784$ with the squared loss on the same data, by Adam, for 60 epochs, and compares.

| **Quantity, at $d = 32$** | **Linear autoencoder** | **Rank-32 projection** |
|---|---:|---:|
| Training error, per pixel | 0.01510 | 0.01501 (the floor) |
| Test error, per pixel | 0.01523 | 0.01519 |
| Principal angles under 2° | 30 of 32 | — |
| $R^2$ of the code regressed on the PCA scores | 0.9979 | — |

The principal angles are between the column space of the decoder's weight matrix and the span of the first 32 right singular vectors: zero if they are the same subspace. Thirty of them are under two degrees. The last two are the exception that confirms step 6: $\sigma_{32}^2$ and $\sigma_{33}^2$, per pixel, are 0.000230 and 0.000227, so the loss barely prefers one direction to the other and neither does the optimiser. The reconstruction is still at the floor, because the loss does not care which of the two it kept. The last row is the uniqueness remark: the code is not the principal components, but a linear map of them explains it almost entirely.

## 5 The undercomplete autoencoder

### 5.1 The model

Two ReLU layers around the bottleneck, and a sigmoid on the output so that a reconstruction is an image in $[0, 1]$:

```
class AutoEncoder(nn.Module):
    def __init__(self, d, hidden=256):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(784, hidden), nn.ReLU(),
                                 nn.Linear(hidden, d))
        self.dec = nn.Sequential(nn.Linear(d, hidden), nn.ReLU(),
                                 nn.Linear(hidden, 784), nn.Sigmoid())

    def forward(self, x):
        return self.dec(self.enc(x))
```

`enc` and `dec` are attributes on purpose: the code is read by calling the encoder alone. At $d = 32$ the model has 419,120 parameters, almost all of them in the two 784-wide layers. The loop is the one of Lecture 8 with `F.mse_loss(out, xb)` in place of the cross-entropy: twenty epochs of Adam at $10^{-3}$, batches of 128, 12,000 images, which a CPU handles. Everything else in the loop — the optimiser, the batches, the evaluation — carries over unchanged, because nothing in it ever depended on the target being a label.

### 5.2 Run it at $d = 32$

| **Model, test error per pixel** | **Error** |
|---|---|
| Rank-32 projection — the best any linear model can do | 0.01519 |
| Linear autoencoder | 0.01523 |
| **Non-linear autoencoder, 419,120 parameters** | **0.01275** |

About a sixth below the floor. That sixth is the quantity the derivation defined: the error a subspace cannot remove, which a curved map can. It is the entire justification for the two ReLU layers. Training error 0.01188 against validation 0.01276: a small gap, so little overfitting, and small because the bottleneck is doing the regulariser's job.

### 5.3 The code size, swept

The same three models at $d = 2, 8, 32, 128$, each scored on the test set, under one recipe — the same epochs, learning rate and width — so that a difference between columns is the code size and not the budget.

| $d$ | **Projection (exact)** | **Linear autoencoder** | **Non-linear autoencoder** |
|---|---|---|---|
| 2 | 0.04610 | 0.04611 | **0.03041** |
| 8 | 0.02662 | 0.02665 | **0.01617** |
| 32 | 0.01519 | 0.01523 | **0.01275** |
| 128 | **0.00641** | 0.00649 | 0.01179 |

The linear network tracks the exact projection at every $d$, as the derivation says it must. The non-linear one wins where the bottleneck binds and, at $d = 128$, *loses*.

**Failure condition — a trained approximation against an exact optimum**

The projection is the exact minimum of its class, from one SVD. The network is a *trained* approximation to the minimum of a larger class, and at $d = 128$ its budget — one hidden layer of 256, twenty epochs — leaves it above what the SVD computes in closed form. More capacity is not more accuracy until the optimiser has found it. The notebook's `try` on this cell widens the hidden layer, and the $d = 128$ row is the one that moves.

### 5.4 What the loss sees, and what it does not

Figure 1 shows eight test garments reconstructed four ways. Rows two and three — the projection and the linear network — are indistinguishable, which is the derivation in pictures. Row four is sharper. Row five, at $d = 2$, is the shape of a kind of garment and nothing of this one.

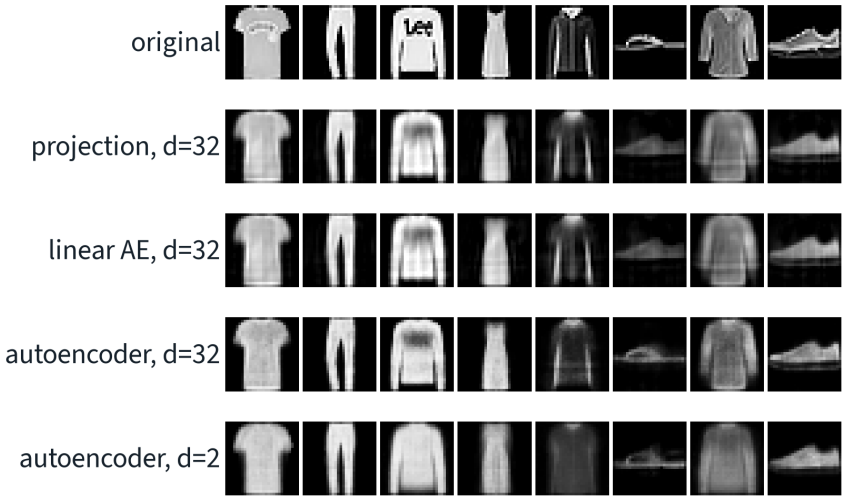



Figure 1: Eight test garments: the originals, the rank-32 projection, the linear autoencoder at 32, the non-linear autoencoder at 32, and the non-linear autoencoder at 2.

Squared error on pixels is a distance between two arrays, and it has no idea which pixels matter. It rewards blur: the average of a class is close, in pixel distance, to every member of the class, so a model short of capacity reconstructs the average and is scored well for it. It is blind to a shift: move a sharp edge one pixel and the error is large; smear it over three and the error is smaller, so the sharper reconstruction is the worse one. It measures pixels, not garments: a missing button costs nothing, a slightly different grey over a whole sleeve costs a lot.

**Failure condition — a loss that rewards the average**

Reconstruction error falling is not the same as reconstructions improving. Look at them. A code chosen by the loss alone is a code tuned to reproduce the common, and the common is what you already knew.

## 6 The code as a representation

## 6.1 The code as a picture

With $d = 2$ the code can be drawn. Every test image becomes a point, and the colour is its label, which the network never saw (Figure 2). Footwear sits apart from clothing, trousers apart from both, and the five torso garments on top of one another: two numbers cannot tell a shirt from a pullover, and the reconstruction error at $d = 2$ already said so.

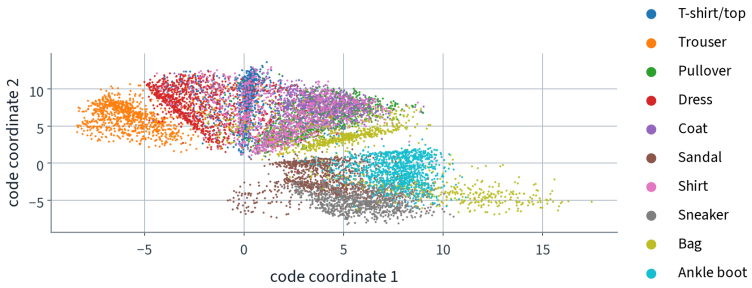



Figure 2: The two-dimensional code of every test image, learned without labels and with the colour added afterwards.

## 6.2 What a representation is for

Thirty-two numbers per garment, computed from the pixels, learned from the pixels. The product questions become geometry in that space.

| Question | Operation on the codes | Scored by |
|---|---|---|
| similar items | nearest neighbours by Euclidean distance | precision at ten, against the labels |
| groups | $k$-means with $k = 10$ | mean purity of the clusters, against the labels |
| the odd ones | reconstruction error as a score | AUROC, against a held-out class |

Every one of them has a baseline that needs no learning: the same operation on the 784 raw pixels, and on the 32 PCA scores. A learned code that cannot beat its own input has learned nothing the question needed.

## 6.3 Similar items: nearest neighbours

The catalogue is the 10,000 test images; a thousand of them, drawn once from a fixed seed, are the queries, each searching the other 9,999 by Euclidean distance. With $\|a-b\|^2 = \|a\|^2 - 2a\cdot b + \|b\|^2$ the thousand searches are one matrix product; a query's own entry is set to infinity so that no image is its own neighbour.

**Definition 6.1** (Precision at ten)**.** *Of the ten images returned for a query, the fraction whose class is the query's, averaged over the queries. The label is read here and nowhere else.*

| Space | Numbers per item | Precision at 10 |
|---|---|---|
| Raw pixels | 784 | 76.71% |
| PCA scores | 32 | 76.47% |
| Linear autoencoder code | 32 | 77.07% |
| **Autoencoder code** | 32 | **77.32%** |

Four spaces within a point of one another. The code did not beat the pixels, and it could not have: nothing in the reconstruction loss is about which garments are *similar*. What it bought is the second column. Swept over the code size, the non-linear code scores 58.18% at $d = 2$, 74.30% at $d = 8$, 77.32% at $d = 32$ and 78.27% at $d = 128$: two numbers lose a fifth of the precision, eight recover most of it, thirty-two match the pixels, and 128 add a point. From eight upwards the representation question is a cost question.

## 6.4 The bill

A nearest-neighbour search compares the query with every item in the catalogue, so the length of the code is the cost of the search and of the index that holds it.

| Per item | Raw pixels | Code | Factor |
|---|---|---|---|
| Numbers stored | 784 | 32 | 24.5 |
| Multiplications per comparison | 784 | 32 | 24.5 |
| Bytes, as float32 | 3,136 | 128 | 24.5 |

At equal precision the last column is the factor on the index's memory and on the query's cost, and for a photograph of real size the 28 by 28 of this dataset becomes thousands of pixels and the factor grows with it. The case for a learned representation in a product is, first of all, this table. Quality is the problem of the lectures that train a representation on pairs.

## 6.5 Groups: $k$-means on the code

Cluster the test set into ten groups with $k$-means, with ten restarts from a fixed seed so that a comparison between spaces is not a comparison between starts, and score each clustering against the labels by the purity of Lecture 6: for each cluster, the share of its members that belong to its commonest class, averaged over the ten clusters.

| Space, $k = 10$ | Numbers per item | Mean purity |
|---|---|---|
| Raw pixels | 784 | 0.6473 |
| PCA scores | 32 | 0.6471 |
| Autoencoder code | 32 | 0.5891 |

Pixels and PCA scores tie, and the learned code loses six points: nothing in the reconstruction loss is about which garments belong together, and here it shows. The structure $k$-means finds is the structure the two-dimensional picture drew — footwear, trousers, bags, torso — and it is in the pixels already. The code carries it in 32 numbers, at a price the audit shows.

## 7 The denoising autoencoder

Corrupt the input, keep the target clean:

```
for xb in batches(X_fit):
    xin  = (xb + 0.3 * torch.randn_like(xb)).clamp(0, 1)   # corrupt the input
    opt.zero_grad()
    loss = F.mse_loss(net(xin), xb)                        # the target is clean
    loss.backward(); opt.step()
```

The second constraint from the table: what comes in is not what must go out, so copying is impossible even with a wide code. The noise is drawn fresh on every batch — the same corruption twenty times over is a fixed dataset, and can be memorised — and it is data augmentation, training on randomly altered copies of each input, with the clean image as the target: a regulariser applied to the training input only.

| **Error against the *clean* test image, per pixel** | **Error** |
|---|---|
| The noisy image itself | 0.05264 |
| The plain autoencoder, fed the noisy image | 0.02408 |
| **The denoising autoencoder, fed the noisy image** | **0.01551** |
| The denoising autoencoder, fed the clean image | 0.01781 |

Three things in four rows. The bottleneck alone removes half the noise — noise does not fit through 32 numbers either. Training on corrupted input removes most of the rest. And the price: on clean input the denoiser is a worse autoencoder, 0.01781 against 0.01275, and its code retrieves slightly worse, 75.87% against 77.32%. A regulariser is a trade: the constraint that stops copying also stops some fitting.

## 8 Anomaly detection by reconstruction error

Train on what the catalogue holds; score every new image by how badly it is reconstructed. An image unlike anything in training should reconstruct badly. No labels in training and no examples of the anomaly needed, which is the whole appeal and the whole risk. To *measure* it, hold one class out of the fit set, train on the other nine for fifteen epochs, score the whole test set — the seen classes included, because a detector is judged on what it flags among the normal items as much as on what it catches — and repeat for each class.

**Definition 8.1** (AUROC)**.** *The area under the ROC curve: the probability that a random held-out image scores above a random seen one. It is 0.5 for a score that carries no information and 1.0 for a perfect separation, and it needs no threshold, so it measures the score rather than one operating point of it.*

| Held-out class | AUROC | Mean error, seen | Mean error, held out |
|---|---|---|---|
| Bag | **0.9382** | 0.01293 | 0.03335 |
| Trouser | 0.8674 | 0.01421 | 0.02695 |
| Sandal | 0.8664 | 0.01484 | 0.03004 |
| Ankle boot | 0.7982 | 0.01440 | 0.02469 |
| Dress | 0.6543 | 0.01386 | 0.01678 |
| Sneaker | 0.6081 | 0.01419 | 0.01534 |
| Pullover | 0.5832 | 0.01404 | 0.01553 |
| T-shirt/top | 0.5549 | 0.01380 | 0.01521 |
| Shirt | 0.5267 | 0.01394 | 0.01455 |
| Coat | **0.5249** | 0.01391 | 0.01345 |

Bags held out are caught; coats held out are not. The coats' mean error, 0.01345, is *below* the seen classes' 0.01391: the detector would flag a random pullover before a coat.

**Failure condition — the score measures imitability, not rarity**

Reconstruction error says how well the training classes can *imitate* an image. A novelty they can imitate — a coat, given pullovers, shirts and T-shirts — is never flagged; an ordinary image they cannot is flagged every time. Whether this detector works is a fact about what the anomalies will look like, and it has to be measured on examples of them.

### 8.1 The three failure conditions, in one place

| Condition | What happens | Where it was measured |
|---|---|---|
| No constraint on the code | the identity is reachable; zero loss, nothing learned | the floor at $d = 784$ is 0 |
| A trained approximation against an exact optimum | at a wide bottleneck the network sits above the SVD | $d = 128$: 0.01179 against 0.00641 |
| Reconstruction error as an anomaly score | an imitable novelty is never flagged | coats: AUROC 0.5249 |

And one that is a property of the loss rather than of the method: squared error rewards the average, so a falling loss is not an improving reconstruction until you have looked.

## 9 The variational autoencoder

Beyond the syllabus, for context.

The autoencoders so far give one code per image and nothing in between: decode a point between two codes and there is no reason the output is a garment. The **variational autoencoder** makes the encoder output a distribution, $\mathcal{N}(\boldsymbol{\mu}(\mathbf{x}), \operatorname{diag}\, \boldsymbol{\sigma}^2(\mathbf{x}))$, samples a code from it, and adds a term that keeps every such distribution close to $\mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$L = \underbrace{\|\mathbf{x} - \hat{\mathbf{x}}\|^2}_{\text{reconstruction}} + \underbrace{\tfrac{1}{2} \sum_{j=1}^{d} \left(\sigma_j^2 + \mu_j^2 - 1 - \log \sigma_j^2\right)}_{\text{KL to the standard normal}} .$$

The second term is a regulariser on the code — the third row of the table of constraints — and it fills the code space: a code drawn from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ decodes to something plausible, which makes the decoder a generator.

A sampled code breaks the chain rule of Lecture 8: there is no derivative of a random draw with respect to the parameters that chose its distribution. The **reparametrisation trick** moves the randomness outside the graph,

$$\mathbf{z} = \boldsymbol{\mu}(\mathbf{x}) + \boldsymbol{\sigma}(\mathbf{x}) \odot \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

so that $\boldsymbol{\varepsilon}$ is drawn first and treated as an input, $\mathbf{z}$ is a deterministic function of $\boldsymbol{\mu}$, $\boldsymbol{\sigma}$ and $\boldsymbol{\varepsilon}$, and reverse mode differentiates through $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ as through any layer. Not examined; it is here because every generative model from Lecture 24 onwards contains this move, and because it is one line.

## 10 Where this leads

| Representation | Trained by | What distances in it mean | Lecture |
|---|---|---|---|
| PCA scores | the SVD | pixel distance, in the best $d$ directions | 6 |
| **autoencoder code** | reconstructing the input | pixel distance, through a curved map, in a basis nobody chose | 11 |
| a classifier's penultimate layer | labels | whatever separates the classes | 13 |
| a dense-retrieval embedding | query–document pairs | relevance | 20 |
| a contrastive embedding | pairs that belong together | similarity, as the pairs defined it | 23 |

Today's code matched the pixels on retrieval and could not beat them, because the loss never saw a pair. The rest of the course is, in part, the story of what happens when it does.

## 11 The notebook

The notebook runs on the CPU, by choice: every network in it has under half a million parameters, and a CPU run is the one that reproduces on the machines students have. It computes the floor from the SVD, trains the linear autoencoder against it and prints the loss, the test error, the principal angles and the $R^2$ of the code on the PCA scores; trains the non-linear autoencoder, sweeps $d$, draws the reconstructions and the two-dimensional code; measures precision at ten and $k$-means in four spaces; trains the denoiser; and trains ten anomaly detectors, one per held-out class. The numbers in these notes are its run.

**What to change**

Set `NOISE = 0` in the denoising section and run again. The denoising rows become the plain autoencoder's, to the digit — they were always the same model with one line changed.

## 12 Design exercise: "see similar" on a fashion marketplace

The lecture closes with a design exercise in the manner of an ML system-design interview, on the product this lecture's method is built for.

**The task.** An online fashion marketplace sells garments listed by thousands of independent brands and shops. The product is its shopping app; the feature is a "see similar" strip under every item page, and a camera search that finds catalogue items like a photo the shopper took. The users are shoppers; the customers are the sellers, who pay a commission on every sale. The business measures the strip's click-through, the conversion it adds, and the return rate of what it sells. Given: millions of seller-uploaded photographs with an unreliable category field, the shoppers' click and purchase logs, and the sellers' stock and price feeds. Where a regulator exists: data protection, for the shoppers' photographs; consumer protection and brand owners, for what "similar" is allowed to show.

1. **Frame it honestly.** The output is a ranked list of catalogue items for one query image; the metric is precision at the strip's length on held-out items, read beside its anchor, the same strip filled from the query's category at random. The label for scoring comes from what shoppers did — items viewed or bought in the same session, cleaned of the strip's own clicks — with the seller's category field as a weak second source. The baseline before any model: the same category, in stock, most popular first. *Pitfall:* reporting the reconstruction error. It is the training signal and the shopper never sees it; a code can reconstruct every photograph beautifully and return the wrong garments.

2. **The photographs are not as alike as the dataset's.** Sellers shoot on different backgrounds, at different sizes, on or off a model, and a code learned from the pixels learns the studio: the strip returns the same seller's other items. Crop to the garment, resize, normalise the background before anything is fitted, and check the strip for seller clumping. A camera-search image is personal data: encode on the device or discard after encoding, and never keep the photo. The same garment is listed by four sellers and in six colours: link variants to one item before splitting. Write down what one item is before the first fit.

3. **Split by item, and by seller.** Hold out whole items — every photograph and variant — and whole sellers for a second check, because the feature has to work for a shop that joined this morning. Choose the code size, the distance and the ranking rules on the validation items; read the test items once. *Pitfall:* a random split over photographs puts the red and the blue of one dress on both sides, the nearest neighbour of a held-out photo is its own item in another colour, and the precision says nothing about a garment the catalogue has never seen.

4. **Learn the code from the catalogue, and size it by the ranking.** Train the autoencoder on the whole catalogue of standardised photographs — no labels needed, so every listing counts — with a convolutional encoder once the photographs are real-sized; read the code from the encoder alone; sweep its size and pick the smallest that holds the validation precision. Denoising and augmentation as the regulariser: the same garment, shifted, in another colour, on another background, must land on the same code. *Pitfall:* choosing the code size by reconstruction error, which falls at every increase forever while the ranking metric flattens early and the index cost does not.

5. **The index.** Encode every catalogue item once, when it is listed, and store the code beside it;

encode the query at request time and compare against stored codes, never against photographs. Exact search is a matrix product at a hundred thousand items; at millions an approximate-neighbour index trades a little recall for a hundred-fold speed, and that recall is measured. Normalise the codes to unit length and compare by cosine. *Pitfall:* a distance nobody chose — the code's coordinates are in a basis the optimiser landed on, and one coordinate with a large range dominates the Euclidean distance.

6. **Similarity is one feature of the ranking.** The strip ranks the nearest hundred by a second model that sees similarity, stock, size availability, price band, the seller's return rate and the shopper's history — a table, and a boosted ensemble on it — with business rules on top: in stock only, the shopper's size, nothing from a seller under review. *Pitfall:* the look-alike under a brand's page. Visual similarity is blind to who made the garment; under a luxury brand's listing the strip shows the imitations that look most like it, the brand — a paying customer — sees its page selling look-alikes, and the counterfeit complaints land with the regulator. Brand and seller constraints belong in the ranking layer, and the similarity model is never the last word.

7. **Screening uploads by reconstruction error.** The same encoder–decoder scores every new upload: a screenshot, a blank, a photograph of a receipt reconstructs badly and goes to a review queue before it is listed. Calibrate the threshold per category on the validation items and measure the queue's precision with the reviewers' decisions. *Pitfall:* the score follows what the catalogue can imitate. A plain photograph of nothing passes; an elaborate, legitimate gown is flagged and its seller waits a day. Route to review, never reject automatically, and read the detector's AUROC on the kinds of upload it is meant to catch.

8. **Release, monitor, retrain.** Release behind an A/B test on live traffic, read on the strip's click-through, the conversion it adds and the return rate of what it sells, with the sellers' exposure held fair across both arms. Monitor the distribution of codes as the catalogue turns over with the seasons; retrain on a schedule and re-encode the whole catalogue in one atomic swap of the index. Keep a randomised slot in the strip, so that the clicks the next model trains on are not only the ones this model chose. *Pitfall:* two encoders in one index. A retrained encoder puts the same garment at a different code, so codes from two versions compare numbers that no longer mean the same thing and the strip fills with garments that are near nothing. Version the encoder with the index.

9. **Ship it.** Listings, then cropped and standardised photographs with variants linked into items, then a split by item and by seller, then an autoencoder on the catalogue with the code sized on validation precision, then normalised codes in a versioned index, then a ranking model with business rules on top, then uploads screened to a review queue, then an A/B test, then retraining, re-encoding and monitoring — each step named for the pitfall it avoids.

## 13 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 12's deck and in the exercise book.

1. A linear autoencoder $784 \to d \to 784$ with squared loss is trained to convergence on centred

data. State the smallest error it can reach, in terms of the singular values of the centred data, and name the subspace through which it reconstructs. [4]

2. Two linear autoencoders are trained from different seeds to the same loss, and their codes for the same image differ. State whether this is a bug, and why. [4]

3. The code size is set equal to the number of pixels and nothing else constrains the network. State what the trained network computes, and what its code is worth as a representation. [3]

4. An anomaly detector scores images by their reconstruction error from an autoencoder trained on nine classes; on the tenth, held out, it scores at chance. Explain the mechanism, and say what would be needed to know in advance whether such a detector works. [5]

5. Nearest-neighbour retrieval in a 32-number autoencoder code scores the same precision at ten as in the 784 raw pixels. State what the code bought, and why it could not have bought more. [4]

## Summary

An autoencoder is a network trained to reproduce its input through a code too small to hold it; the bottleneck is a constraint on what the network can reach, and without one the identity is reachable. A linear autoencoder with squared loss cannot beat the projection onto the principal subspace and reaches it: the floor is $\sum_{j>d} \sigma_j^2$, the subspace is Lecture 6's, and the code is unique only up to a change of basis — measured, the floor is 0.01501 and the trained network reaches 0.01510, with thirty of 32 principal angles under two degrees. The non-linear model sits a sixth below the floor at $d = 32$ and above the exact projection at $d = 128$, because a trained approximation is not an exact optimum. Read as a representation, 32 numbers retrieve and cluster as well as 784 pixels, 77.32% against 76.71% on precision at ten, and no better, at a twenty-fourfold saving: the loss never saw a pair. Corrupting the input is a regulariser that stops copying and costs some fitting. Reconstruction error catches bags held out at an AUROC of 0.9382 and misses coats at 0.5249, because it measures imitability, not rarity. The closing design exercise built a marketplace's "see similar" feature in nine steps — a ranking metric against its anchor, standardised photographs, a split by item, a code sized by precision, a normalised and versioned index, business rules above similarity, a review queue, an A/B test — each named for the pitfall it avoids.